

AI / ML SCREENING KIT

RAG · Agentic AI · Orchestration · LLM

20 multiple-choice questions with fixed answers

For non-technical screeners

Target profile: 4–6 years experience | Duration: 20–25 minutes

Candidate: _____ Date: _____
Screener: _____ Score: _____ / 20

How to use this kit

This kit is designed so that a screener with no AI background can run it. Every question has one correct answer — you are simply matching the letter the candidate gives against the answer printed below the options.

Before you start

Ask this first: “Which framework have you used for building AI agents?”

If the answer is LangGraph or LangChain, ask all 20 questions.

If the answer is CrewAI, Google ADK, OpenAI Agents SDK or PydanticAI, skip Section 3 and score out of 15. This is not a negative — it is simply a different toolchain.

During the call

- Read the question, then all four options in order A, B, C, D. Do not shuffle them.
- Record the letter the candidate gives. Do not hint, explain, or react to the answer.
- One repeat of the question is allowed if the candidate asks for it.
- Allow 20–30 seconds of thinking time. A pause before answering is a good sign, not a bad one.
- If the candidate would rather explain than pick a letter, allow it. A correct explanation counts as correct — write “explained” beside it. This is often a stronger candidate, not a weaker one.

Two things that matter more than the total

Standout questions. Five questions are marked with a star. Getting these right indicates the candidate has actually operated these systems rather than only read about them. Note them separately on the scorecard — they carry more weight than the raw total.

Hard-fail rule. If the candidate gets both Section 1 Q2 and Section 2 Q1 wrong, they are not hands-on with RAG or agents, whatever the total score says.

Your job is to filter out clear mismatches, not to make the final decision. When in doubt, forward the candidate to the technical round with your notes.

Section 1 · RAG

RAG means making an AI answer from the company's own documents rather than from general knowledge. It is the most common enterprise AI project today.

Q1. You switch to a new embedding model. Documents are already stored using the old one. What must you do?

- A. Nothing — only new documents need it
- B. Re-embed and re-index all existing documents
- C. Retrain the language model
- D. Restart the server

Answer: B ★ standout — Picking A means they have never migrated a live system. Mixing embedding models breaks search silently, with no error.

Q2. The correct answer is in the documents, but the chatbot answered wrongly. Most likely cause?

- A. The model isn't powerful enough
- B. The temperature is too low
- C. The right chunk was never retrieved
- D. The documents were uploaded too recently

Answer: C ★ standout — The best RAG filter in the kit. Picking A is blaming the model for what is actually a retrieval failure.

Q3. Users search exact part codes like “XR-4471” and get irrelevant results, though the code is in the documents. Fix?

- A. Increase the temperature
- B. Add hybrid search — keyword plus vector
- C. Use a bigger language model
- D. Delete and re-upload the documents

Answer: B — Vector search matches meaning, not exact strings, so it is poor at codes and IDs.

Q4. Which metric checks whether the answer is supported by the retrieved documents, not invented?

- A. Response latency
- B. Faithfulness
- C. Uptime
- D. Token cost

Answer: B — Picking A, C or D means they have never evaluated a RAG system.

Q5. The leave policy PDF was updated last week, but the chatbot still quotes the old rules. What was missed?

- A. Retraining the model
- B. Re-indexing the updated document
- C. Increasing the temperature
- D. Adding more tools

Answer: B — The index still holds the old text.

Section 2 · Agentic AI

An agent is AI that decides its own next steps and takes action using tools — not just a chatbot that replies.

Q1. An agent runs 12 steps and costs far more than expected. Why does agent cost grow so fast?

- A. The framework charges a fee per tool call
- B. The full conversation history is resent at every step, so tokens compound
- C. The vector database charges per step
- D. Agents always use the most expensive model

Answer: B — *A production-experience question. Anyone who has run an agent and seen the bill knows this.*

Q2. An agent calls a payment API. It times out, so the agent retries. What property stops a double charge?

- A. Rate limiting
- B. Streaming
- C. Idempotency
- D. Caching

Answer: C ★ standout — *Pronounced “eye-dem-POT-en-see”. Proves agents touching real production systems, not demos.*

Q3. An agent keeps repeating the same two steps and never finishes. What stops it?

- A. A maximum step limit
- B. A higher temperature
- C. More tools
- D. A bigger context window

Answer: A — *Also accept “recursion limit” or “timeout”.*

Q4. An agent has 25 tools and often picks the wrong one. What do you try first?

- A. Split it into a multi-agent system
- B. Improve the tool names and descriptions, and reduce the tool list
- C. Raise the temperature
- D. Increase the step limit

Answer: B — *Jumping straight to multi-agent is the buzzword answer and usually makes it worse.*

Q5. An agent gave a wrong result after 9 steps. Fastest way to find which step failed?

- A. Re-run it with a bigger model
- B. Increase the step limit
- C. Lower the temperature and retry
- D. Review the trace — every step and tool call logged in an observability tool

Answer: D — *A, B and C are guesswork; only D is debugging. Good follow-up: which tool? (LangSmith, Langfuse, Phoenix)*

Section 3 · Orchestration — LangGraph

Skip this section if the candidate named CrewAI, Google ADK, the OpenAI Agents SDK or PydanticAI, and score out of 15 instead. Not a negative.

Q1. A LangGraph StateGraph is built from which two components?

- A. Prompts and parsers
- B. Documents and indexes
- C. Nodes and edges
- D. Tools and memories

Answer: C — Nodes do the work, edges decide what runs next. If wrong, they have not written LangGraph code.

Q2. Refunds under ₹5,000 auto-process; above that, route to a human. What makes that decision in LangGraph?

- A. A conditional edge
- B. An output parser
- C. A retriever
- D. A callback handler

Answer: A — Accept “conditional routing” or “router function”.

Q3. An agent pauses for approval overnight. The server restarts. What lets it resume from that exact step?

- A. The vector store
- B. The checkpoint / persistence layer
- C. The output parser
- D. The embedding model

Answer: B ★ standout — Only people who have built real human-in-the-loop approval flows know this.

Q4. Core difference between a LangChain chain and a LangGraph graph?

- A. LangGraph only works with OpenAI models
- B. A chain runs straight through; a graph supports loops with shared state
- C. LangChain is the newer replacement
- D. They are the same product

Answer: B — This is why LangGraph exists — an agent must be able to loop.

Q5. A process runs identically every time: extract invoice → validate → write to database. Agent or workflow?

- A. An autonomous agent
- B. A multi-agent system
- C. A fixed workflow
- D. A RAG pipeline

Answer: C ★ standout — Knowing when NOT to use an agent is one of the clearest seniority signals.

Section 4 · LLM Fundamentals

Basic fluency with how large language models behave, cost and are controlled.

Q1. Fine-tuning is best used for?

- A. Adding facts that change frequently, like daily prices
- B. Changing the model's style, tone and behaviour
- C. Making the model run faster
- D. Reducing storage cost

Answer: B — *Picking A means they think fine-tuning is a way to load data into a model.*

Q2. A client wants brand tone and today's live prices. Correct approach?

- A. Fine-tune the model on both
- B. Use RAG for both
- C. Tone via prompt or fine-tuning; live prices via RAG or an API call
- D. Not possible today

Answer: C — *Catches anyone who got Q1 right by luck.*

Q3. One response in twenty breaks the JSON parser downstream. Fix?

- A. Add "please return valid JSON" to the prompt
- B. Lower the top-k
- C. Use structured output / JSON schema mode with validation
- D. Increase the context window

Answer: C — *Prompt-only JSON is exactly what breaks at scale.*

Q4. Compliance needs the same answer every time. Which setting?

- A. Temperature 1
- B. Temperature 0
- C. A bigger context window
- D. More retries

Answer: B — *Bonus if they add the caveat: near-deterministic, not an absolute guarantee.*

Q5. The same 30 questions are asked daily and a long system prompt goes out on every call. Costs tripled. Best fix?

- A. Prompt caching plus response caching
- B. Move to a larger model
- C. Add more agents
- D. Increase the chunk size

Answer: A — *Also accept routing easy questions to a cheaper, smaller model.*

Answer key

Tear this page off, or keep it out of view if you are sharing your screen.

Section	Q1	Q2	Q3	Q4	Q5
RAG	B ★	C ★	B	B	B
Agentic AI	B	C ★	A	B	D
Orchestration	C	A	B ★	B	C ★
LLM	B	C	C	B	A

Scoring

Score	Reading	Action
17 – 20	Strong, hands-on at depth	Fast-track; flag to hiring manager
13 – 16	Solid 4–6 year practitioner	Forward to technical round
9 – 12	Real but shallow exposure	Forward with your notes
Under 9	Not hands-on	Reject for a GenAI role

If Section 3 was skipped, use these bands instead: 13+ strong · 10–12 forward · 7–9 forward with notes · under 7 reject.

Standout markers

Note these separately. They carry more weight than the total score.

Question	What getting it right proves
RAG Q1	Has actually migrated a live RAG system
RAG Q2	Has debugged RAG, not only built it
Agentic Q2	Agents touching real production systems
Orchestration Q3	Built genuine human-in-the-loop approval flows
Orchestration Q5	Mature architectural judgement — knows when not to use an agent

Hard-fail rule

Wrong on both RAG Q2 and Agentic Q1 → the candidate is not hands-on with RAG or agents, regardless of the total score.

Notes for the technical panel

Communication, honesty, red flags, anything worth passing on:

Vocabulary can be crammed in a week; judgement cannot. A candidate who explains their reasoning well but misses some terminology is usually the better hire.

Glossary for the screener

Plain-English meaning of terms you may hear during the call. You do not need to understand these to run the kit.

Term	Plain English
RAG	Making the AI answer from your own documents rather than general knowledge
Embedding	Text converted into numbers that capture its meaning
Chunk / chunking	Splitting long documents into small pieces before storing them
Vector database	Storage that finds similar meanings fast — Pinecone, Qdrant, pgvector, ChromaDB, FAISS
Re-indexing	Refreshing the stored documents when the source changes
Hybrid search	Keyword search plus meaning search combined; needed for codes and exact names
Faithfulness	Whether the answer is actually supported by the retrieved documents
Hallucination	The AI confidently making something up
Agent	AI that decides its own steps and uses tools to take action
Tool / function calling	Letting the AI run a real function or API
Workflow vs agent	Workflow = fixed steps every time. Agent = decides the steps itself
Human-in-the-loop	A person approves or corrects the AI's action before it runs
Idempotency	Repeating the same request has the same effect as sending it once
Trace / observability	A record of every step an agent took, used for debugging
LangChain	Framework for building LLM applications
LangGraph	Framework for building agents as graphs; supports loops and pausing
LangSmith / Langfuse	Platforms for tracing, debugging and evaluating AI applications
Checkpoint	Saves the agent's state so a paused run can resume later
Node / edge	Node does the work; edge decides what runs next
CrewAI / Google ADK / OpenAI Agents SDK / PydanticAI	Alternative agent frameworks — equally valid choices
Token	Chunk of text; LLM cost is charged per token
Temperature	Randomness setting; 0 gives consistent, repeatable output
Fine-tuning	Further training to change style and behaviour — not a way to add facts
Structured output	Forcing the model to return valid JSON reliably
Pydantic / PydanticAI	The Python library used to define and validate structured output schemas; PydanticAI is an agent framework built on it
Prompt caching	Reusing a repeated long prompt to cut cost